

Project 3 - Part 2

Mandip Chaudhary (17)

2025-04-16

Part2

Use this link <https://kateto.net/netscix2016.html> to replicate all the SNA results and interpret them carefully.

Reading the datasets::

First data bases consists of Network of hyperlinks and news sources among media organisations.

```
media_nodes <- read.csv("Dataset1-Media-Example-NODES.csv", header=T, as.is=T)
media_edges <- read.csv("Dataset1-Media-Example-EDGES.csv", header=T, as.is=T)
head(media_nodes)
```

##	id	media	media.type	type.label	audience.size
## 1	s01	NY Times	1	Newspaper	20
## 2	s02	Washington Post	1	Newspaper	25
## 3	s03	Wall Street Journal	1	Newspaper	30
## 4	s04	USA Today	1	Newspaper	32
## 5	s05	LA Times	1	Newspaper	20
## 6	s06	New York Post	1	Newspaper	50

```
head(media_edges)
```

##	from	to	weight	type
## 1	s01	s02	10	hyperlink
## 2	s01	s02	12	hyperlink
## 3	s01	s03	22	hyperlink
## 4	s01	s04	21	hyperlink
## 5	s04	s11	22	mention
## 6	s05	s15	21	mention

```
nrow(media_nodes); length(unique(media_nodes$id))
```

```
## [1] 17
```

```
## [1] 17
```

```
nrow(media_edges); nrow(unique(media_edges[,c("from", "to")]))
```

```
## [1] 52
```

```
## [1] 49
```

Remedying the 3 non unique links.

```
media_edges <- aggregate(media_edges[,3], media_edges[, -3], sum)
```

```
media_edges <- media_edges[order(media_edges$from, media_edges$to),]
```

```
colnames(media_edges)[4] <- "weight"
```

```
rownames(media_edges) <- NULL
```

Database 2 consists of network of links between media venues and consumers:

```
media_user_nodes <- read.csv("Dataset2-Media-User-Example-NODES.csv", header=T, as.is=T)
```

```
media_user_adj_matrix <- read.csv("Dataset2-Media-User-Example-EDGES.csv", header=T, , as.is=T)
```

```
head(media_user_nodes)
```

```
##      id  media media.type media.name audience.size
## 1 s01    NYT           1 Newspaper           20
## 2 s02    WaPo           1 Newspaper           25
## 3 s03    WSJ            1 Newspaper           30
## 4 s04    USAT           1 Newspaper           32
## 5 s05 LATimes           1 Newspaper           20
## 6 s06    CNN            2         TV           56
```

```
head(media_user_adj_matrix)
```

```
##      X U01 U02 U03 U04 U05 U06 U07 U08 U09 U10 U11 U12 U13 U14 U15 U16 U17 U18
## 1 s01   1   1   1   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
## 2 s02   0   0   0   1   1   0   0   0   0   0   0   0   0   0   0   0   0   0
## 3 s03   0   0   0   0   0   1   1   1   1   0   0   0   0   0   0   0   0   0
## 4 s04   0   0   0   0   0   0   0   0   1   1   1   0   0   0   0   0   0   0
## 5 s05   0   0   0   0   0   0   0   0   0   0   1   1   1   0   0   0   0   0
## 6 s06   0   0   0   0   0   0   0   0   0   0   0   0   1   1   0   0   1   0
##      U19 U20
## 1      0   0
## 2      0   1
## 3      0   0
## 4      0   0
## 5      0   0
## 6      0   0
```

Links is an adjacency matrix. So, we make it a matrix:

```
media_user_adj_matrix <- as.matrix(media_user_adj_matrix)

dim(media_user_adj_matrix)
```

```
## [1] 10 21
```

```
dim(media_user_nodes)
```

```
## [1] 30 5
```

Nodes has 30 values, first 10 of media companies and other 20 of the users.

IGraph

```
library(igraph)

media_network <- graph_from_data_frame(d=media_edges, vertices=media_nodes, directed=T)
```

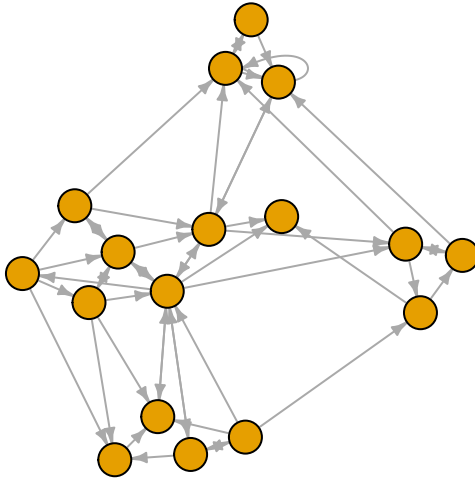
```
E(media_network)      # The edges of the media network
```

```
## + 49/49 edges from ee26606 (vertex names):
## [1] s01->s02 s01->s03 s01->s04 s01->s15 s02->s01 s02->s03 s02->s09 s02->s10
## [9] s03->s01 s03->s04 s03->s05 s03->s08 s03->s10 s03->s11 s03->s12 s04->s03
## [17] s04->s06 s04->s11 s04->s12 s04->s17 s05->s01 s05->s02 s05->s09 s05->s15
## [25] s06->s06 s06->s16 s06->s17 s07->s03 s07->s08 s07->s10 s07->s14 s08->s03
## [33] s08->s07 s08->s09 s09->s10 s10->s03 s12->s06 s12->s13 s12->s14 s13->s12
## [41] s13->s17 s14->s11 s14->s13 s15->s01 s15->s04 s15->s06 s16->s06 s16->s17
## [49] s17->s04
```

```
V(media_network)      # The vertices of the media network
```

```
## + 17/17 vertices, named, from ee26606:
## [1] s01 s02 s03 s04 s05 s06 s07 s08 s09 s10 s11 s12 s13 s14 s15 s16 s17
```

```
plot(media_network, edge.arrow.size=.4,vertex.label=NA)
```



Removing loops::

```
media_network <- simplify(media_network, remove.multiple = F, remove.loops = T)
```

Converting into graph using the links matrix.

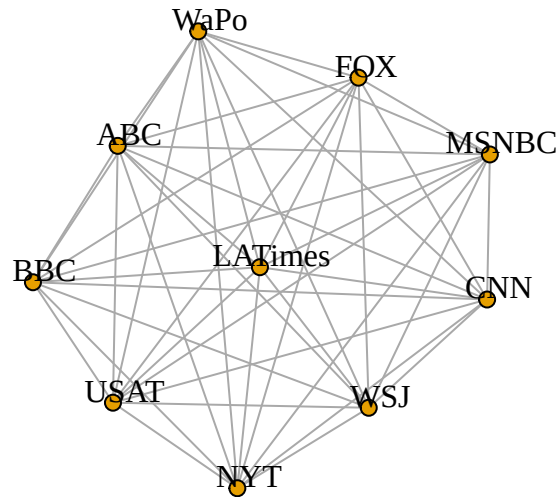
```
bipartite_network <- graph_from_biadjacency_matrix(media_user_adj_matrix)
```

```
table(V(bipartite_network)$type)
```

```
##
## FALSE  TRUE
##    10    21
```

Plot of Media Organizations

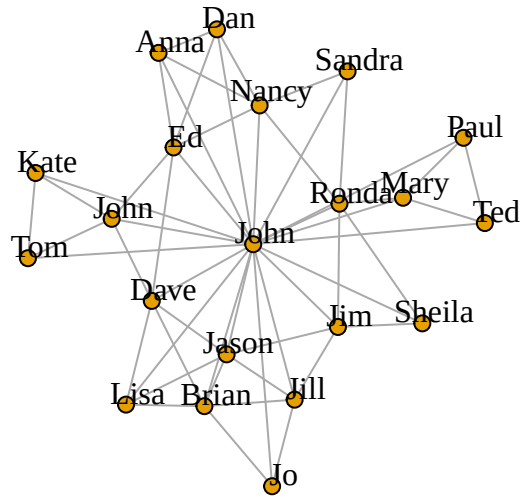
```
bipartite_proj <- bipartite_projection(bipartite_network)
plot(bipartite_proj$proj1, vertex.label.color="black", vertex.label.dist=1,
     vertex.size=7, vertex.label=media_user_nodes$media[!is.na(media_user_nodes$media.type)])
```



Interpretation: This visualization reveals the network of media organizations based on shared user connections. The graph shows distinct clusters of media outlets, such as one group containing BBC, NYT, USAT, and LA Times interconnected through BBC, and another cluster featuring CNN, FOX, and MSNBC. These groupings suggest that media organizations within each cluster share similar audience bases and potentially similar content focuses. However, it's important to note that bipartite projections can simplify complex relationships, making them most suitable for initial exploratory analysis rather than definitive conclusions about media relationships.

Plot of Media Consumers

```
plot(bipartite_proj$proj2, vertex.label.color="black", vertex.label.dist=1,
     vertex.size=7, vertex.label=media_user_nodes$media[is.na(media_user_nodes$media.type)])
```

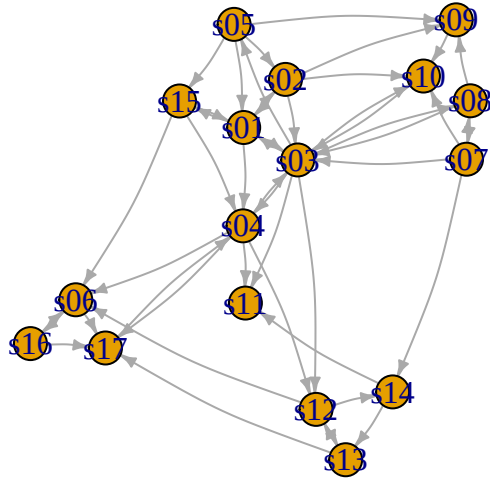


Interpretation: This visualization represents connections between media consumers based on their shared media consumption patterns. The network reveals how different users are linked through their common choices of media sources. While this bipartite projection provides insights into user clustering and media consumption similarities, it's important to note that it represents a simplified view of the actual consumption patterns, as the original bipartite relationships between users and media sources are compressed into user-to-user connections.

Plotting networks with igraph

For Database 1, we explore different visualization options:

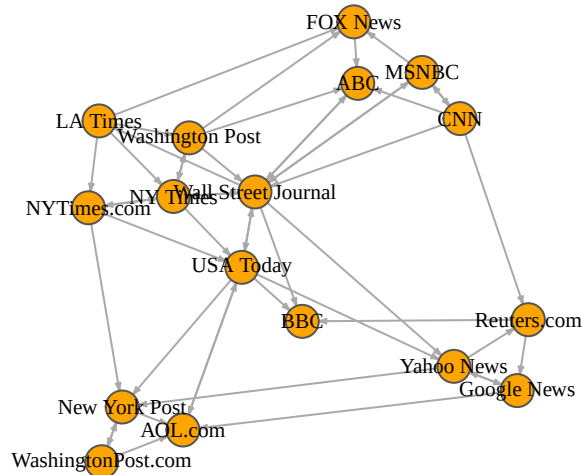
```
plot(media_network, edge.arrow.size=.4, edge.curved=.1)
```



Adding media organization labels with improved styling:

```
set.seed(42)

plot(media_network, edge.arrow.size=.2, edge.curved=0,
     vertex.color="orange", vertex.frame.color="#555555",
     vertex.label=V(media_network)$media, vertex.label.color="black",
     vertex.label.cex=.7)
```



Interpretation: This network visualization reveals the complex web of hyperlink connections between media organizations. Unlike the simplified bipartite representation, this directed graph shows both reciprocal and one-way relationships between organizations. Notable patterns include the Wall Street Journal's position as a frequent link target from other outlets, while organizations like Google, Yahoo, and Reuters tend to be more prolific in providing outbound links to other media sources. These patterns suggest different roles within the digital media ecosystem, with some organizations serving as content hubs while others act more as content distributors.

We now change the size of node depending on the number of audience in the graph.

```
# Customize node appearance based on media attributes:

colrs <- c("gray50", "tomato", "gold") # Colors for different media types

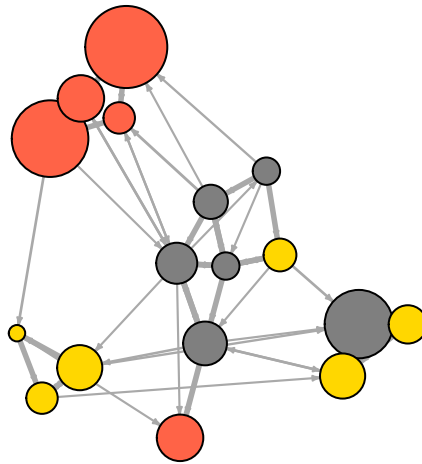
# Apply visual attributes to nodes
V(media_network)$color <- colrs[V(media_network)$media.type]
V(media_network)$size <- V(media_network)$audience.size*0.7
V(media_network)$label.color <- "black"
V(media_network)$label <- NA

# Customize edge appearance
E(media_network)$width <- E(media_network)$weight/6
E(media_network)$arrow.size <- .2
E(media_network)$edge.color <- "gray80"
E(media_network)$width <- 1+E(media_network)$weight/12

# Plot the network with the applied styling
```



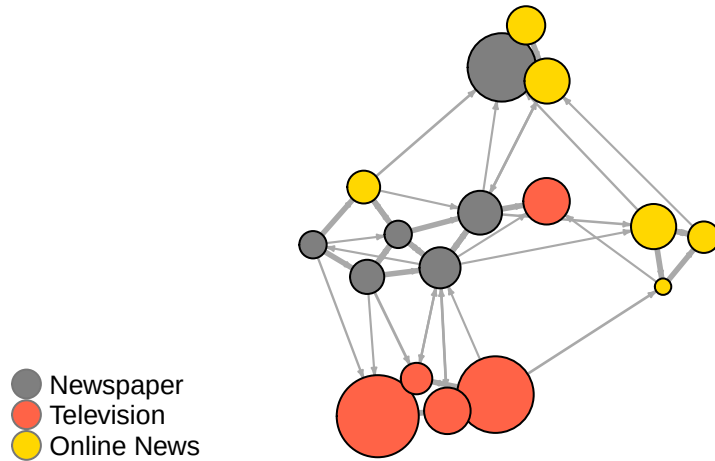
```
plot(media_network)
```



Display the network with a legend indicating media types:

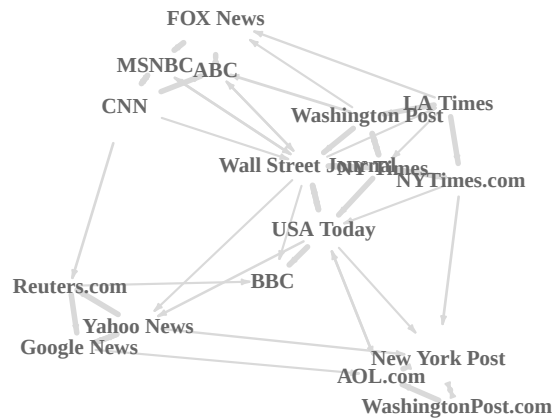
```
plot(media_network)

legend("bottomleft", c("Newspaper", "Television", "Online News"),
      pch=21, col="#777777", pt.bg=colrs, pt.cex=2,
      cex=.8, bty="n", ncol=1)
```



Interpretation: This visualization emphasizes the varying audience sizes across different media types. Television outlets generally demonstrate the largest audience reach, as indicated by their node sizes. While newspapers and online news sources typically show smaller audience sizes, there's a notable exception where one newspaper achieves television-level audience numbers. This suggests that while traditional media formats like television tend to reach larger audiences, individual outlets can transcend these typical patterns, possibly through effective digital strategy or strong brand recognition.

```
plot(media_network, vertex.shape="none", vertex.label=V(media_network)$media,
     vertex.label.font=2, vertex.label.color="gray40",
     vertex.label.cex=.7, edge.color="gray85")
```



Network Layouts and different types of layouts

Here, we mainly observe and study different ways we can layout a network.

- We observe that if we do not save the layouts, each run will result in different layouts.
- Some of these layouts may not be useful for large data but work well for smaller data for visualization like the circle, sphere, line layouts.
- Trying different layouts will help us manipulate the results to how we want to present it. This can be misleading depending on how we use it.

```
# Create a sample network for layout demonstrations
set.seed(123)
layout_network <- sample_pa(80) # Create a preferential attachment network with 80 nodes

# Set node visual properties
V(layout_network)$size <- 8
V(layout_network)$frame.color <- "white"
V(layout_network)$color <- "orange"
V(layout_network)$label <- ""

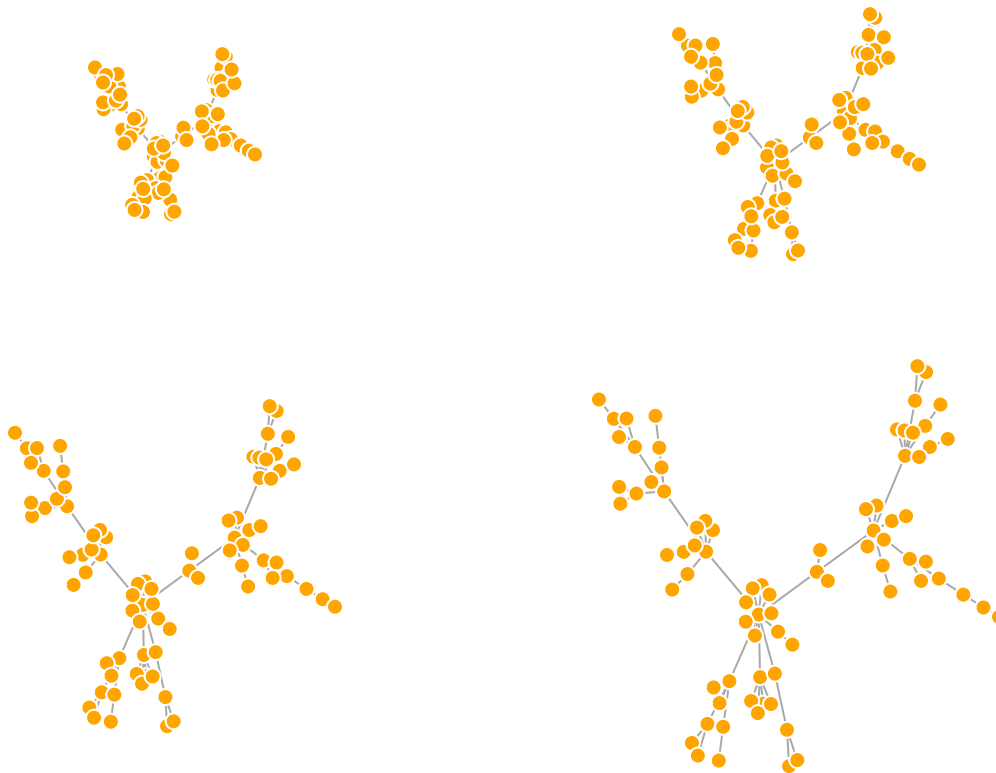
# Set edge properties
E(layout_network)$arrow.mode <- 0
```

```

# Create normalized FR layout coordinates
l <- layout_with_fr(layout_network)
l <- norm_coords(l, ymin=-1, ymax=1, xmin=-1, xmax=1)

# Plot at different scales
par(mfrow=c(2,2), mar=c(0,0,0,0))
plot(layout_network, rescale=F, layout=l*0.4)
plot(layout_network, rescale=F, layout=l*0.6)
plot(layout_network, rescale=F, layout=l*0.8)
plot(layout_network, rescale=F, layout=l*1.0)

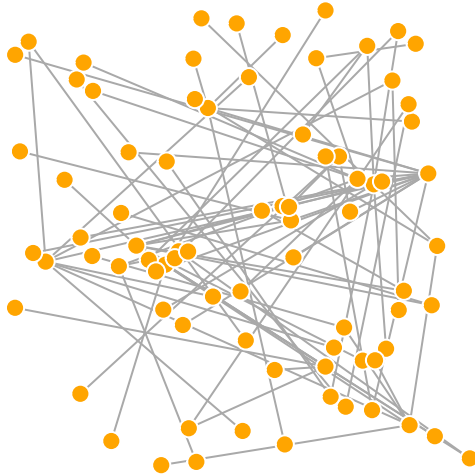
```



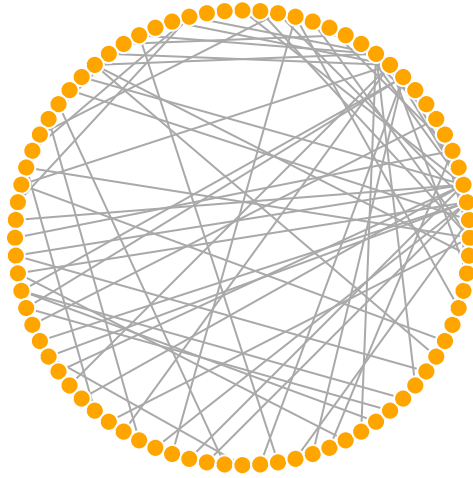
```

# Try random layout
plot(layout_network, layout=layout_randomly)

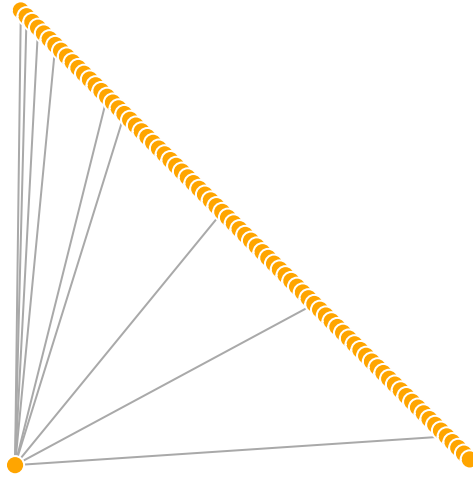
```



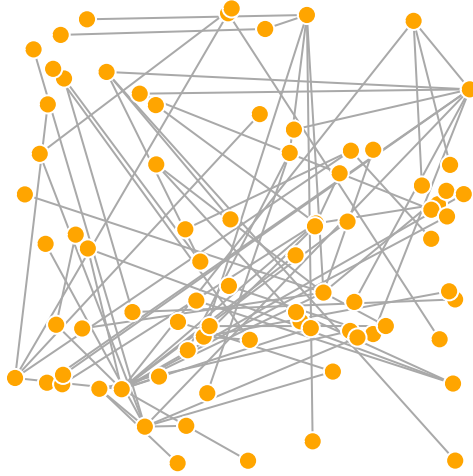
```
# Try circular layout  
l <- layout_in_circle(layout_network)  
plot(layout_network, layout=l)
```



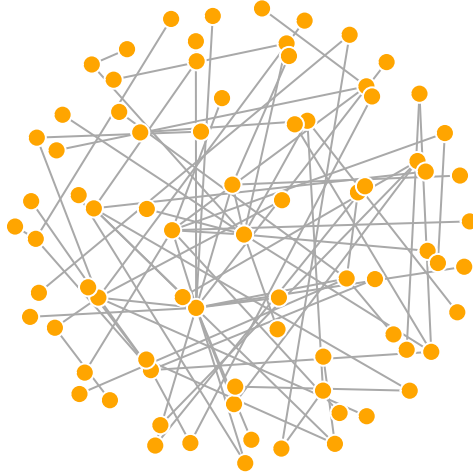
```
# Create a layout with numbered vertex positions  
l <- cbind(1:vcount(layout_network), c(1, vcount(layout_network):2))  
plot(layout_network, layout=l)
```



```
# Try another random layout  
l <- layout_randomly(layout_network)  
plot(layout_network, layout=l)
```



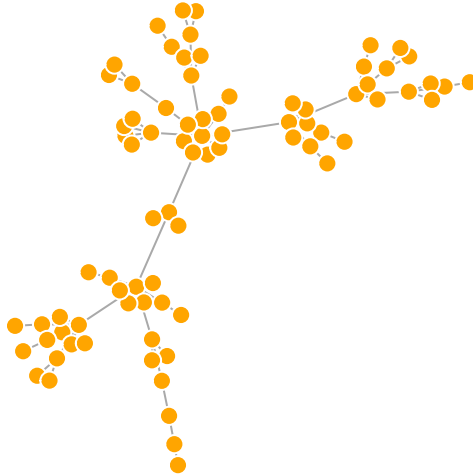
```
# Try spherical layout  
l <- layout_on_sphere(layout_network)  
plot(layout_network, layout=l)
```

Fruchterman-Reingold Algorithm

Apply and save the Fruchterman-Reingold layout

```
l <- layout_with_fr(layout_network)
plot(layout_network, layout=l)
```

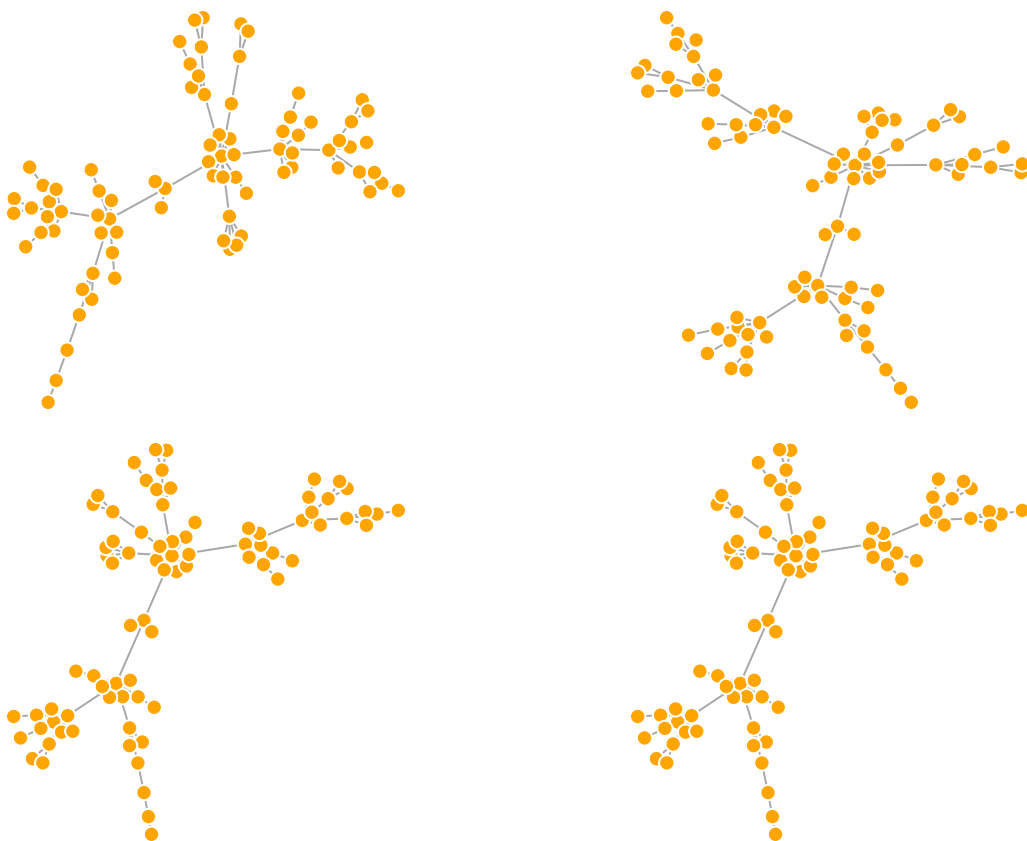


Compare Different FR Layout Runs

```
# Show how FR layout varies between runs
par(mfrow=c(2,2), mar=c(0,0,0,0))  # 2x2 plot grid

# Generate two new random FR layouts
plot(layout_network, layout=layout_with_fr)
plot(layout_network, layout=layout_with_fr)

# Plot saved layout twice for comparison
plot(layout_network, layout=1)
plot(layout_network, layout=1)
```



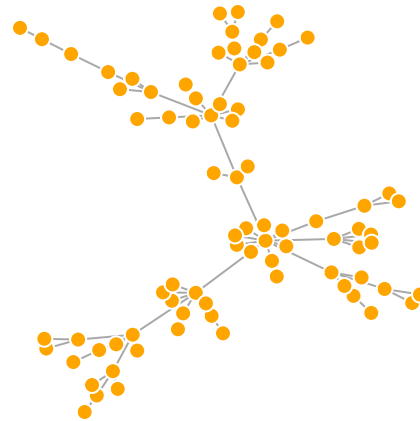
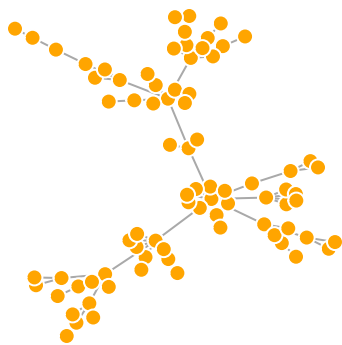
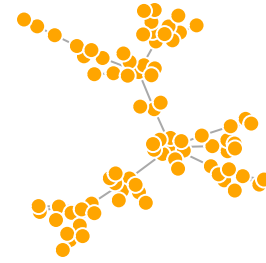
Compare Different Layout Scales:

```
l <- layout_with_fr(layout_network)

l <- norm_coords(l, ymin=-1, ymax=1, xmin=-1, xmax=1)

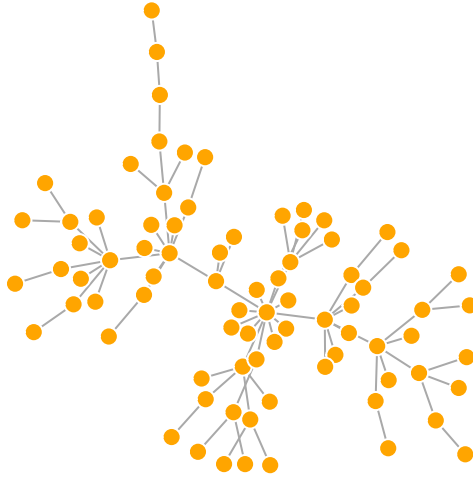
par(mfrow=c(2,2), mar=c(0,0,0,0))

plot(layout_network, rescale=F, layout=l*0.4)
plot(layout_network, rescale=F, layout=l*0.6)
plot(layout_network, rescale=F, layout=l*0.8)
plot(layout_network, rescale=F, layout=l*1.0)
```



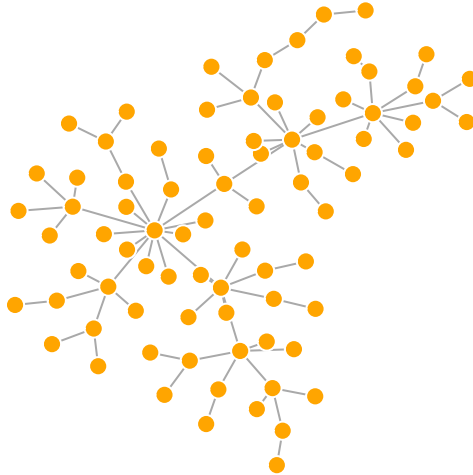
Kamada Kawai Algorithm:

```
# Apply Kamada-Kawai layout  
l <- layout_with_kk(layout_network)  
plot(layout_network, layout=l)
```



LGL algorithm:

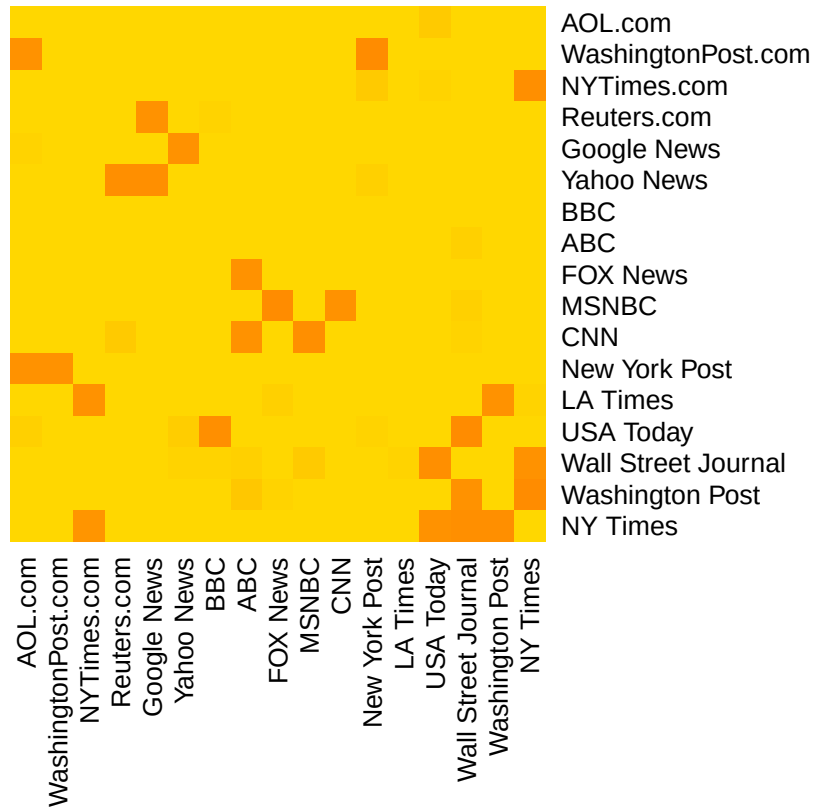
```
# Try LGL layout  
plot(layout_network, layout=layout_with_lgl)
```



Other ways to represent data:

Heatmap:

```
netm <- get.adjacency(media_network, attr="weight", sparse=F)
colnames(netm) <- V(media_network)$media
rownames(netm) <- V(media_network)$media
palf <- colorRampPalette(c("gold", "dark orange"))
heatmap(netm[,17:1], Rowv = NA, Colv = NA, col = palf(100),
        scale="none", margins=c(10,10) )
```



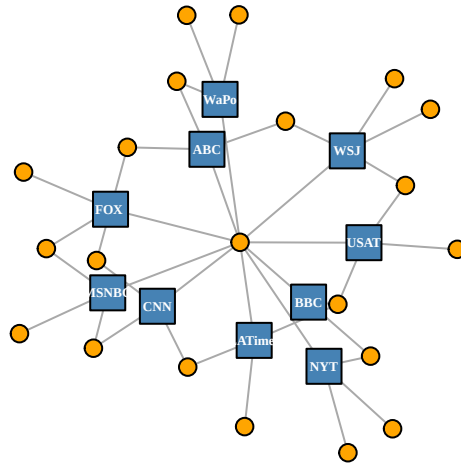
Interpretation: We observe the relations of media orgs with each other. We observe that the clusters formed in network graph will have higher degree of relation shown in heatmap.

Two mode network:

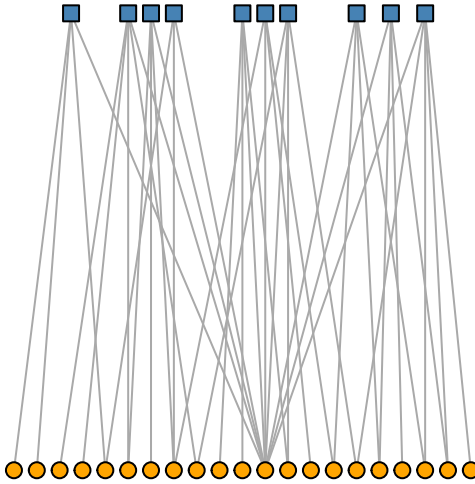
```
V(bipartite_network)
```

```
## + 31/31 vertices, from c2883e7:
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31
```

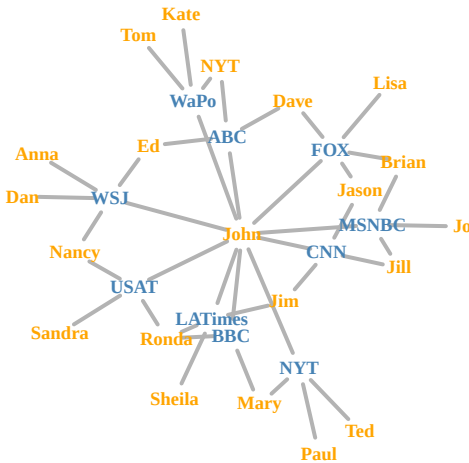
```
V(bipartite_network)$color <- c("steel blue", "orange")[V(bipartite_network)$type+1]
V(bipartite_network)$shape <- c("square", "circle")[V(bipartite_network)$type+1]
V(bipartite_network)$label <- ""
V(bipartite_network)$label[V(bipartite_network)$type==F] <- media_user_nodes$media[V(bipartite_network)$type+1]
V(bipartite_network)$label.cex=.4
V(bipartite_network)$label.font=2
plot(bipartite_network, vertex.label.color="white", vertex.size=(2-V(bipartite_network)$type)*8)
```



```
plot(bipartite_network, vertex.label=NA, vertex.size=7, layout=layout_as_bipartite)
```

```
plot(bipartite_network, vertex.shape="none", vertex.label=media_user_nodes$media,
     vertex.label.color=V(bipartite_network)$color, vertex.label.font=2.5,
     vertex.label.cex=.6, edge.color="gray70", edge.width=2)
```



Interpretation:: Here, we give names to the use nodes and media orgs. We use two mode to combine two graphs to help us visualize their relation. Here, we have for users and media channels. We observe how some users use multiple media sources whereas some are content with only one media source. The media source the users use can be used to group users based on their interests, political views, their location, etc.

Network and Node Descriptives:

Density:

It is the ratio of actual edges to possible edges. It measures network sparsity (e.g., dense = many connections, sparse = few). It scales poorly for large networks.

```
edge_density(media_network, loops=F)
```

```
## [1] 0.1764706
```

```
ecount(media_network)/(vcount(media_network)*(vcount(media_network)-1)) #for a directed network
```

```
## [1] 0.1764706
```

Reciprocity:

Proportion of mutual (bidirectional) edges in directed networks.

- High reciprocity: Balanced relationships (e.g., mutual friendships).
- Low reciprocity: Hierarchical/parasitic interactions (e.g., Twitter follows).

```
reciprocity(media_network)
```

```
## [1] 0.4166667
```

```
dyad_census(media_network) # Mutual, asymmetric, and null node pairs
```

```
## $mut
## [1] 10
##
## $asym
## [1] 28
##
## $null
## [1] 98
```

```
2*dyad_census(media_network)$mut/ecount(media_network) # Calculating reciprocity
```

```
## [1] 0.4166667
```

Transitivity

It is probability that two neighbors of a node are connected.

- High transitivity: Tight-knit communities (e.g., friend groups).
- Low transitivity: Star-like or tree-like structures.

```
transitivity(media_network, type="global") # net is treated as an undirected network
```

```
## [1] 0.372549
```

```
transitivity(as_undirected(media_network, mode="collapse")) # same as above
```

```
## [1] 0.372549
```

```
transitivity(media_network, type="local")
```

```
##      s01      s02      s03      s04      s05      s06      s07      s08
## 0.6000000 0.6000000 0.2500000 0.3333333 0.5000000 0.4000000 0.3333333 0.3333333
##      s09      s10      s11      s12      s13      s14      s15      s16
## 0.3333333 0.5000000 0.3333333 0.3000000 0.3333333 0.1666667 0.5000000 1.0000000
##      s17
## 0.3333333
```

```
triad_census(media_network) # for directed networks
```

```
## [1] 244 241 80 13 11 27 15 22 4 1 8 4 4 3 3 0
```

Diameter:

diameter is the longest geodesic distance (length of the shortest path between two nodes)

```
diameter(media_network, directed=F, weights=NA) ## steps when no weights
```

```
## [1] 4
```

```
diameter(media_network, directed=F)
```

```
## [1] 28
```

```
diam <- get_diameter(media_network, directed=T)  
diam
```

```
## + 7/17 vertices, named, from e6ea5c8:  
## [1] s12 s06 s17 s04 s03 s08 s07
```

```
class(diam)
```

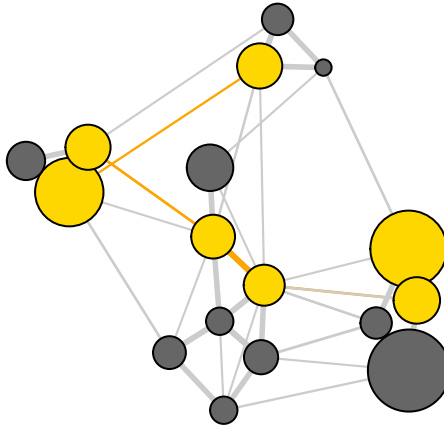
```
## [1] "igraph.vs"
```

```
as.vector(diam)
```

```
## [1] 12 6 17 4 3 8 7
```

The yellow color nodes are the nodes along the diameter.

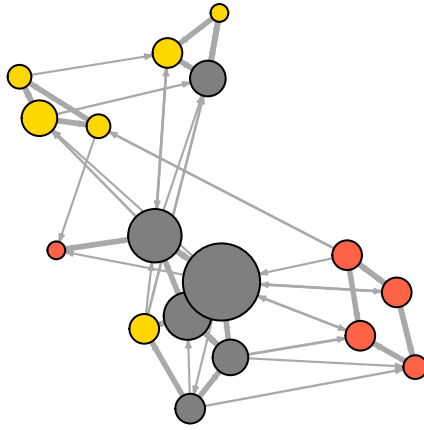
```
vcol <- rep("gray40", vcount(media_network))  
vcol[diam] <- "gold"  
ecol <- rep("gray80", ecoun(media_network))  
ecol[E(media_network, path=diam)] <- "orange"  
# E(media_network, path=diam) finds edges along a path, here 'diam'  
plot(media_network, vertex.color=vcol, edge.color=ecol, edge.arrow.mode=0)
```



Node Degrees:

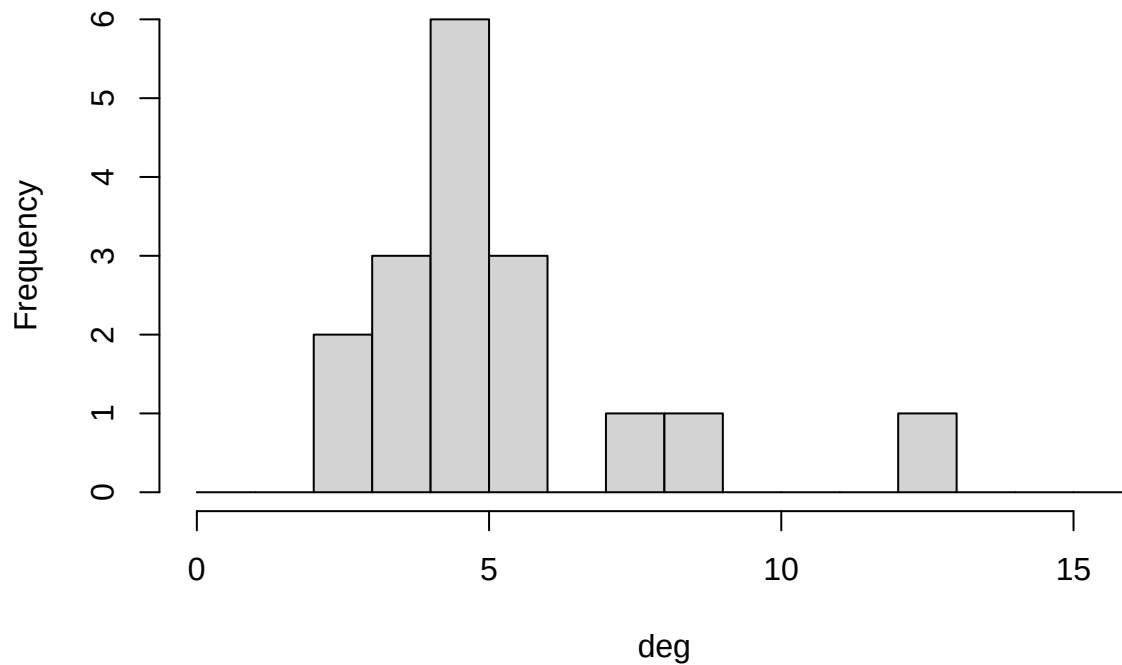
Size of node here represents the total number of ins and outs edges for each node.

```
deg <- degree(media_network, mode="all")  
plot(media_network, vertex.size=deg*3)
```



```
hist(deg, breaks=1:vcount(media_network)-1, main="Histogram of node degree")
```

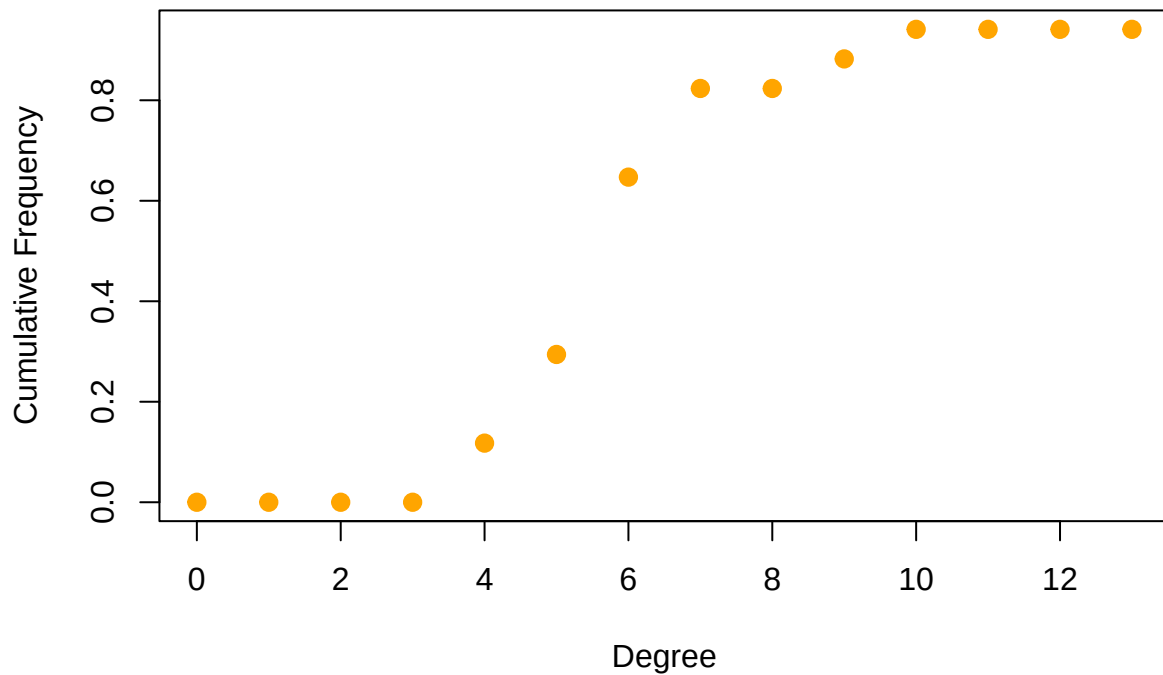
Histogram of node degree



Interpretation:: We observe that most nodes have degrees 3-6 and few have degrees 8,9 and 13.

Degree Distribution:

```
deg.dist <- degree_distribution(media_network, cumulative=T, mode="all")  
  
plot( x=0:max(deg), y=1-deg.dist, pch=19, cex=1.2, col="orange",  
      xlab="Degree", ylab="Cumulative Frequency")
```



We observe how curve is steeper around 6 so there are more frequent nodes with degree 6.

Centrality and Centralization::

Degree:

Number of connections for each nodes, useful for identifying influencers.

```
degree(media_network, mode="in")
```

```
## s01 s02 s03 s04 s05 s06 s07 s08 s09 s10 s11 s12 s13 s14 s15 s16 s17
##  4  2  6  4  1  4  1  2  3  4  3  3  2  2  2  1  4
```

```
centr_degree(media_network, mode="in", normalized=T)
```

```
## $res
## [1] 4 2 6 4 1 4 1 2 3 4 3 3 2 2 2 1 4
##
## $centralization
## [1] 0.1985294
##
## $theoretical_max
## [1] 272
```


Closeness:

It is average distance to others. The closer ones spread information in the network more.

```
closeness(media_network, mode="all", weights=NA)
```

```
##          s01          s02          s03          s04          s05          s06          s07
## 0.03333333 0.03030303 0.04166667 0.03846154 0.03225806 0.03125000 0.03030303
##          s08          s09          s10          s11          s12          s13          s14
## 0.02857143 0.02564103 0.02941176 0.03225806 0.03571429 0.02702703 0.02941176
##          s15          s16          s17
## 0.03030303 0.02222222 0.02857143
```

```
centr_clo(media_network, mode="all", normalized=T)
```

```
## $res
## [1] 0.5333333 0.4848485 0.6666667 0.6153846 0.5161290 0.5000000 0.4848485
## [8] 0.4571429 0.4102564 0.4705882 0.5161290 0.5714286 0.4324324 0.4705882
## [15] 0.4848485 0.3555556 0.4571429
##
## $centralization
## [1] 0.3753596
##
## $theoretical_max
## [1] 7.741935
```

Betweenness:

It is useful for finding nodes on shortest paths useful for routing.

```
betweenness(media_network, directed=T, weights=NA)
```

```
##          s01          s02          s03          s04          s05          s06
## 24.0000000  5.8333333 127.0000000  93.5000000  16.5000000  20.3333333
##          s07          s08          s09          s10          s11          s12
##  1.8333333 19.5000000  0.8333333 15.0000000  0.0000000 33.5000000
##          s13          s14          s15          s16          s17
## 20.0000000  4.0000000  5.6666667  0.0000000 58.5000000
```

```
edge_betweenness(media_network, directed=T, weights=NA)
```

```
## [1] 10.833333 11.333333  8.333333  9.500000  4.000000 12.500000  3.000000
## [8]  2.333333 24.000000 16.000000 31.500000 32.500000  9.500000  6.500000
## [15] 23.000000 65.333333 11.000000  6.500000 18.000000  8.666667  5.333333
## [22] 10.000000  6.000000 11.166667 15.000000 21.333333 10.000000  2.000000
## [29]  1.333333  4.500000 11.833333 16.833333  6.833333 16.833333 31.000000
## [36] 17.000000 18.000000 14.500000  7.500000 28.500000  3.000000 17.000000
## [43]  5.666667  9.666667  6.333333  1.000000 15.000000 74.500000
```

```
centr_betw(media_network, directed=T, normalized=T)
```

```
## $res
## [1] 24.0000000  5.8333333 127.0000000  93.5000000  16.5000000  20.3333333
## [7]  1.8333333 19.5000000  0.8333333  15.0000000  0.0000000  33.5000000
## [13] 20.0000000  4.0000000  5.6666667  0.0000000  58.5000000
##
## $centralization
## [1] 0.4460938
##
## $theoretical_max
## [1] 3840
```

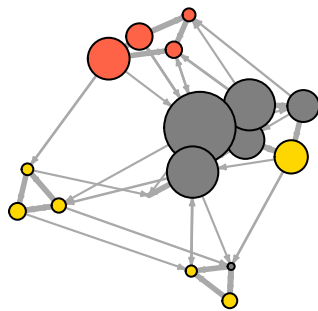
Hubs and Authorities:

The hubs and authorities algorithm developed by Jon Kleinberg was initially used to examine web pages. Hubs were expected to contain catalogs with a large number of outgoing links; while authorities would get many incoming links from hubs, presumably because of their high-quality relevant information.

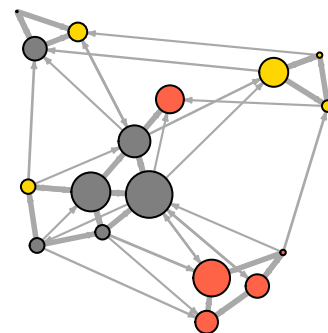
```
hs <- hub_score(media_network, weights=NA)$vector
as <- authority_score(media_network, weights=NA)$vector

par(mfrow=c(1,2))
plot(media_network, vertex.size=hs*50, main="Hubs")
plot(media_network, vertex.size=as*30, main="Authorities")
```

Hubs



Authorities



Distance and Paths:

```
mean_distance(media_network, directed=F)
```

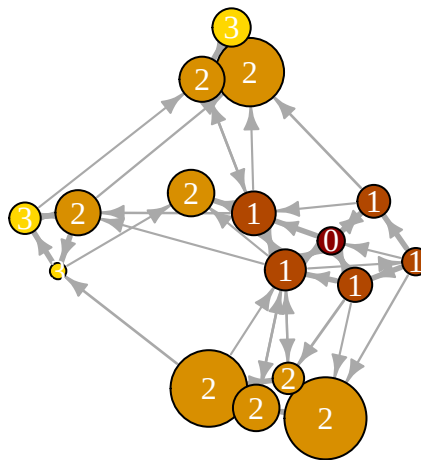
```
## [1] 6.948529
```

```
mean_distance(media_network, directed=T)
```

```
## [1] 27.55078
```

we will get the distance of every media from the New York Times.

```
dist.from.NYT <- distances(media_network, v=V(media_network)[media=="NY Times"], to=V(media_network), w  
# Set colors to plot the distances:  
oranges <- colorRampPalette(c("dark red", "gold"))  
col <- oranges(max(dist.from.NYT)+1)  
col <- col[dist.from.NYT+1]  
plot(media_network, vertex.color=col, vertex.label=dist.from.NYT, edge.arrow.size=.6,  
      vertex.label.color="white")
```



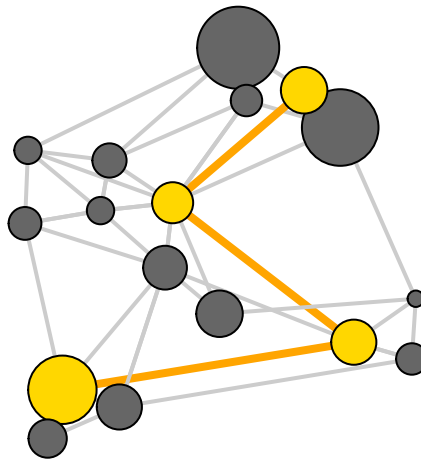
the shortest path between MSNBC and the New York Post

```

news.path <- shortest_paths(media_network,
                             from = V(media_network)[media=="MSNBC"],
                             to   = V(media_network)[media=="New York Post"],
                             output = "both") # both path nodes and edges

# Generate edge color variable to plot the path:
ecol <- rep("gray80", ecount(media_network))
ecol[unlist(news.path$sepath)] <- "orange"
# Generate edge width variable to plot the path:
ew <- rep(2, ecount(media_network))
ew[unlist(news.path$sepath)] <- 4
# Generate node color variable to plot the path:
vcol <- rep("gray40", vcount(media_network))
vcol[unlist(news.path$vpath)] <- "gold"
plot(media_network, vertex.color=vcol, edge.color=ecol,
      edge.width=ew, edge.arrow.mode=0)

```

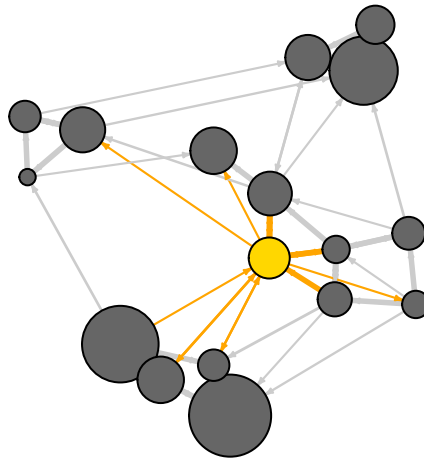


Identify the edges going into or out of a vertex, for instance the WSJ.

```

inc.edges <- incident(media_network, V(media_network)[media=="Wall Street Journal"], mode="all")
# Set colors to plot the selected edges.
ecol <- rep("gray80", ecount(media_network))
ecol[inc.edges] <- "orange"
vcol <- rep("grey40", vcount(media_network))
vcol[V(media_network)$media=="Wall Street Journal"] <- "gold"
plot(media_network, vertex.color=vcol, edge.color=ecol)

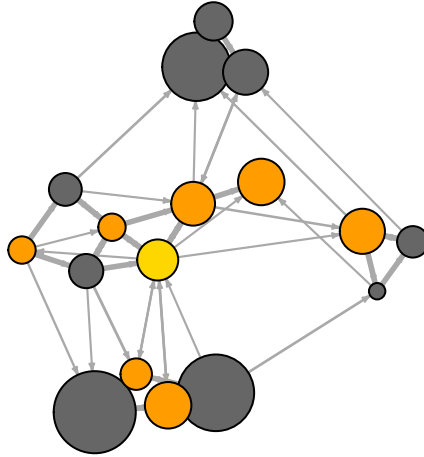
```



We can also identify the immediate neighbors of a vertex, say WSJ. The nodes to which the node edges are going out.

```
neigh.nodes <- neighbors(media_network, V(media_network)[media=="Wall Street Journal"], mode="out")
# Set colors to plot the neighbors:
vcol[neigh.nodes] <- "#ff9d00"

plot(media_network, vertex.color=vcol)
```



Special operators for the indexing of edge sequences: `%-%`, `%->%`, `%<-%` * `E(network)[X %-% Y]` selects edges between vertex sets X and Y, ignoring direction * `E(network)[X %->% Y]` selects edges from vertex sets X to vertex set Y * `E(network)[X %<-% Y]` selects edges from vertex sets Y to vertex set X

select edges from newspapers to online sources

```
E(media_network)[ V(media_network)[type.label=="Newspaper"] %->% V(media_network)[type.label=="Online"]
```

```
## + 7/48 edges from e6ea5c8 (vertex names):
```

```
## [1] s01->s15 s03->s12 s04->s12 s04->s17 s05->s15 s06->s16 s06->s17
```

Subgroups and Committees:

Here, we make the graph undirected. Different ways of doing that are:

- We can create an undirected link between any pair of connected nodes (mode="collapse")
- Create undirected link for each directed one in the network, potentially ending up with a multiplex graph (mode="each")
- Create undirected link for each symmetric link in the graph (mode="mutual").

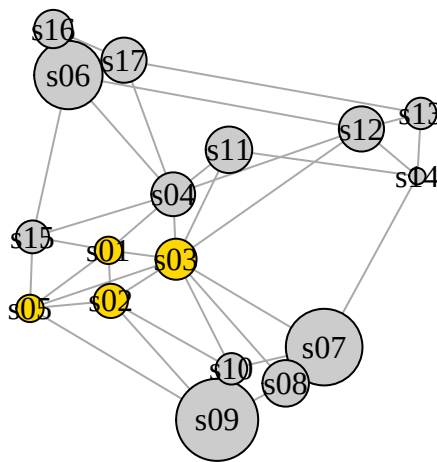
The ties A -> B and B -> A ties are collapsed into a single undirected link. The 'weight' of the links should be summed, and all other edge attributes ignored and dropped.

```
net.sym <- as.undirected(media_network, mode= "collapse",
edge.attr.comb=list(weight="sum", "ignore"))
```

Cliques:

cliques are complete subgraphs of undirected graph.

```
vcol <- rep("grey80", vcount(net.sym))
vcol[unlist(largest_cliques(net.sym))] <- "gold"
plot(as.undirected(net.sym), vertex.label=V(net.sym)$name, vertex.color=vcol)
```

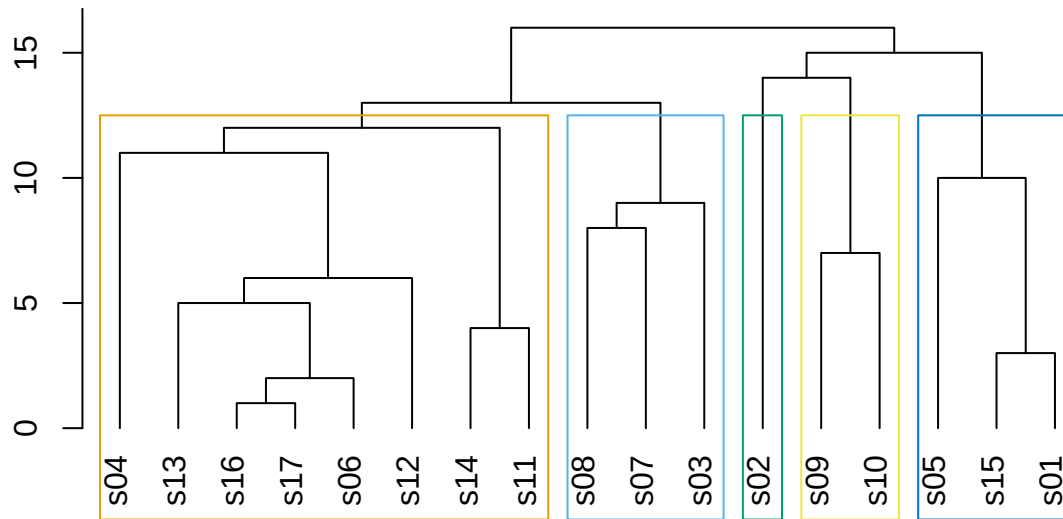


Community Detection:

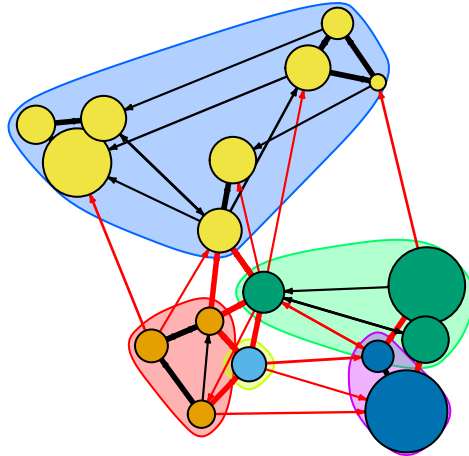
A number of algorithms aim to detect groups that consist of densely connected nodes with fewer connections across groups.

Community detection based on edge betweenness (Newman-Girvan)

```
ceb <- cluster_edge_betweenness(media_network)
plot_dendrogram(ceb, mode="hclust")
```



```
plot(ceb, media_network)
```

```
class(ceb)
```

```
## [1] "communities"
```

```
length(ceb)      # number of communities
```

```
## [1] 5
```

```
membership(ceb) # community membership for each node
```

```
## s01 s02 s03 s04 s05 s06 s07 s08 s09 s10 s11 s12 s13 s14 s15 s16 s17
##   1   2   3   4   1   4   3   3   5   5   4   4   4   4   1   4   4
```

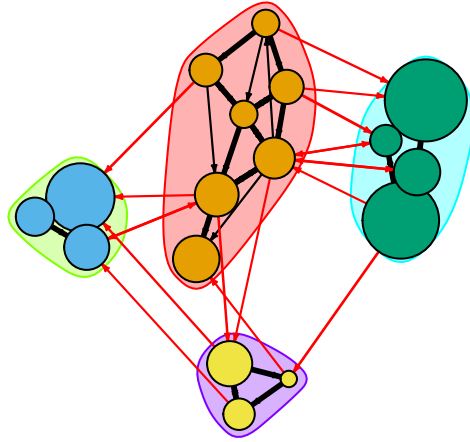
```
modularity(ceb) # how modular the graph partitioning is
```

```
## [1] 0.2970913
```

Community detection based on based on propagating labels

```
clp <- cluster_label_prop(media_network)
```

```
plot(clp, media_network)
```

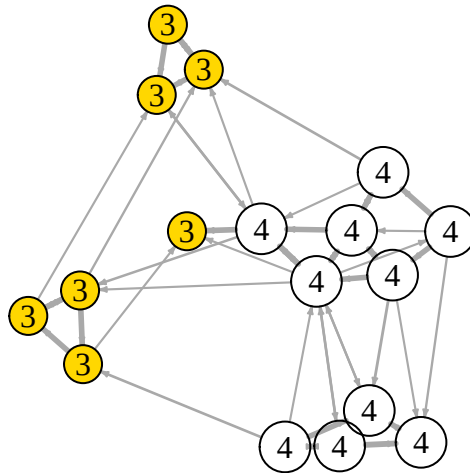


K-core decomposition

The k -core is the maximal subgraph in which every node has degree of at least k . The result here gives the coreness of each vertex in the network. A node has coreness D if it belongs to a D -core but not to $(D+1)$ -core.

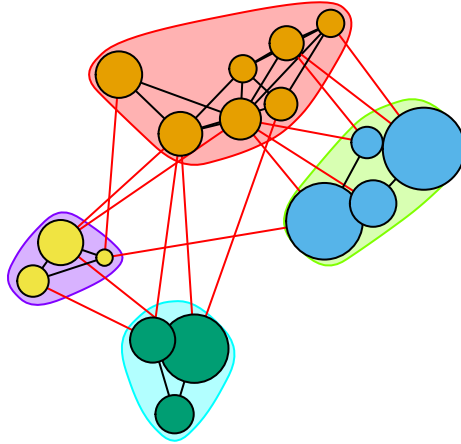
```
kc <- coreness(media_network, mode="all")

plot(media_network, vertex.size=kc*6, vertex.label=kc, vertex.color=colrs[kc])
```



Community detection based on greedy optimization of modularity

```
cfg <- cluster_fast_greedy(as.undirected(media_network))  
plot(cfg, as.undirected(media_network))
```



Assortativity and Homophily

Assortativity: Tendency of nodes to link to similar nodes (by degree, attribute). Homophily: the tendency of nodes to connect to others who are similar on some variable.

- `assortativity_nominal()` is for categorical variables (labels)
- `assortativity()` is for ordinal and above variables
- `assortativity_degree()` checks assortativity in node degrees

```
assortativity_nominal(media_network, V(media_network)$media.type, directed=F)
```

```
## [1] 0.1715568
```

```
assortativity(media_network, V(media_network)$audience.size, directed=F)
```

```
## [1] -0.1102857
```

```
assortativity_degree(media_network, directed=F)
```

```
## [1] -0.009551146
```